

Aula 11

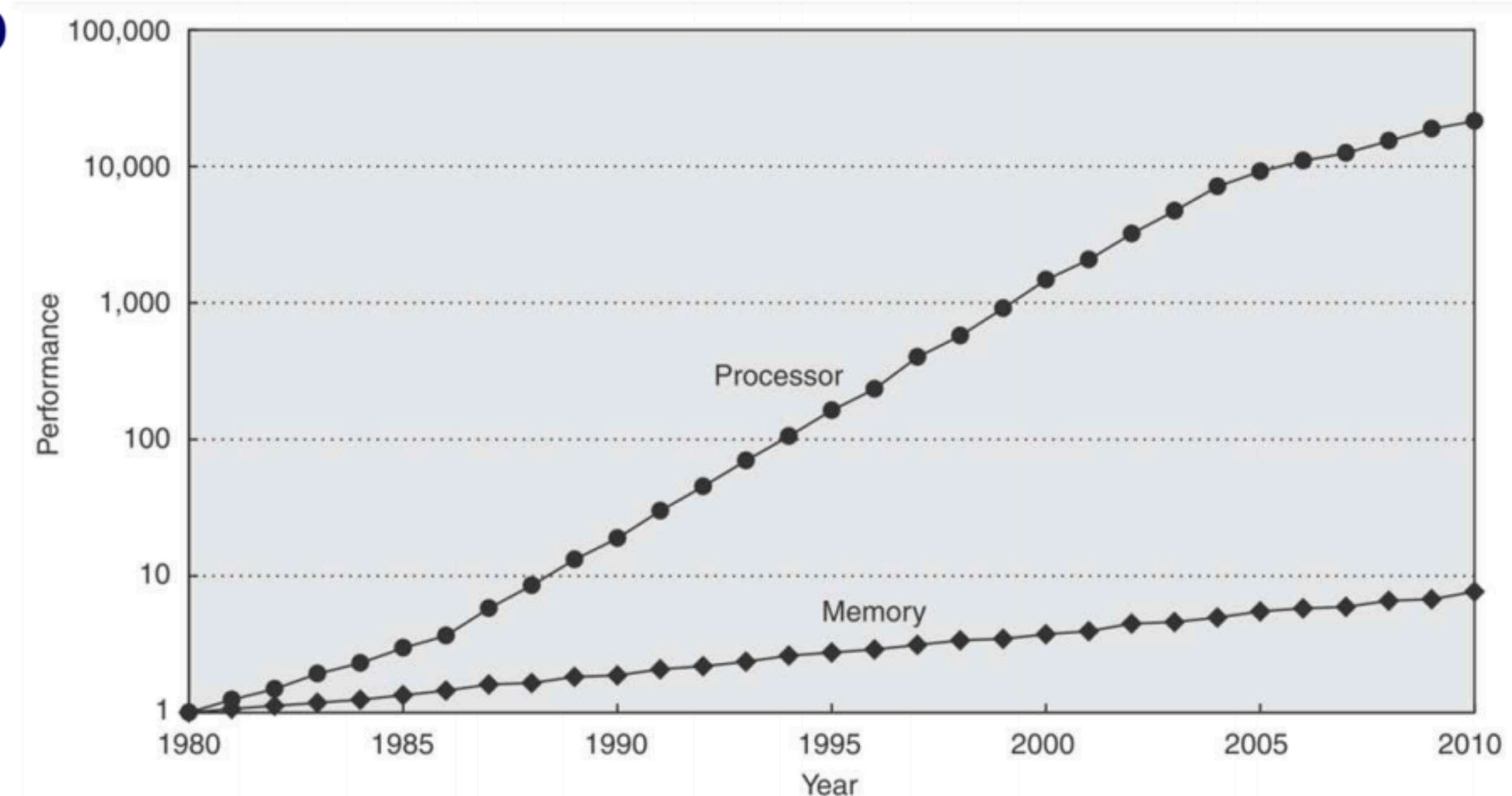
- Organização da memória de um sistema computacional
- Hierarquia do sistema de memória
- Localidade temporal e espacial
- A memória cache; princípio de funcionamento
- Organização
 - Cache com mapeamento associativo
 - Cache com mapeamento direto
 - Cache com mapeamento parcialmente associativo
- Políticas de substituição de blocos. Políticas de escrita.

José Luís Azevedo, Bernardo Cunha, Tomás O. Silva, P. Bartolomeu

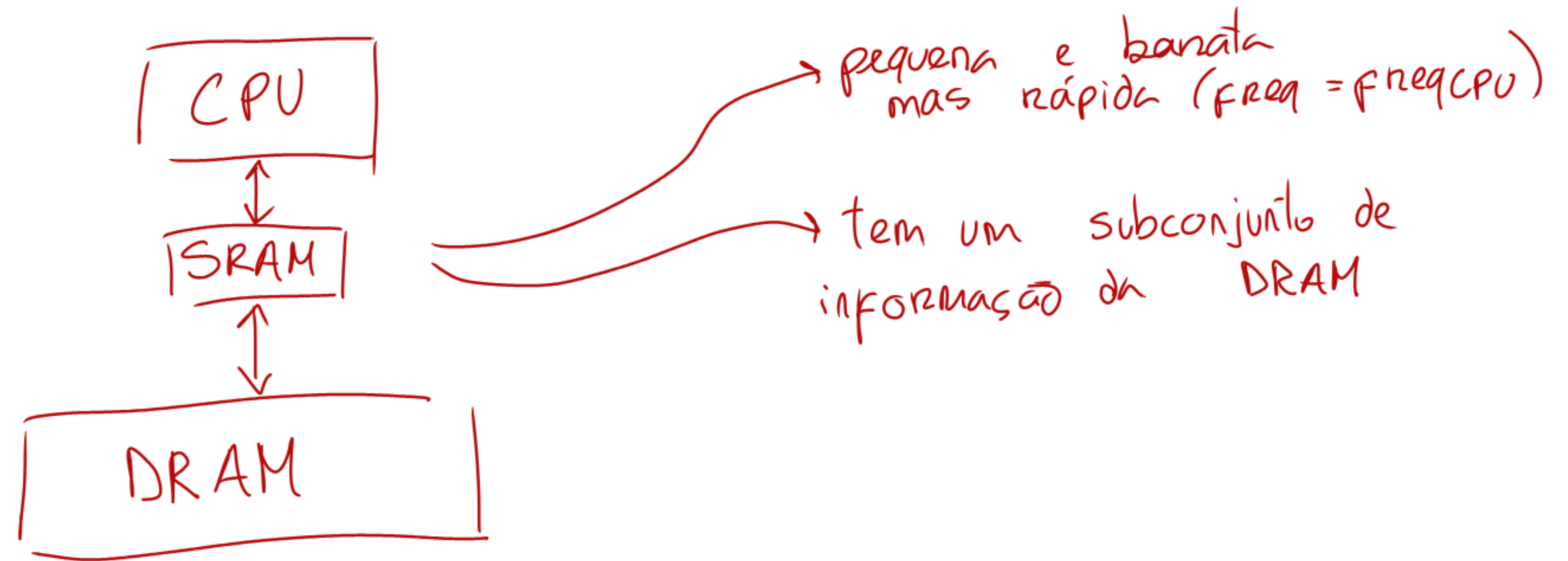
Introdução

- O processador necessita de um fluxo constante de instruções e dados para manter o desempenho

- A diferença entre o desempenho do processador e da memória (DRAM) tem vindo a aumentar



- Solução:
 - Guardar a informação mais vezes utilizada pelo CPU numa memória rápida (*static* RAM) de pequena capacidade
 - Aceder raramente à memória principal (mais lenta) para obter a informação em falta (apenas quando necessário)
 - Transferir blocos de informação da memória principal para a memória rápida
- Conceito: **cache**
 - A cache permite que o processador aceda rapidamente a dados e instruções, melhorando o desempenho do sistema

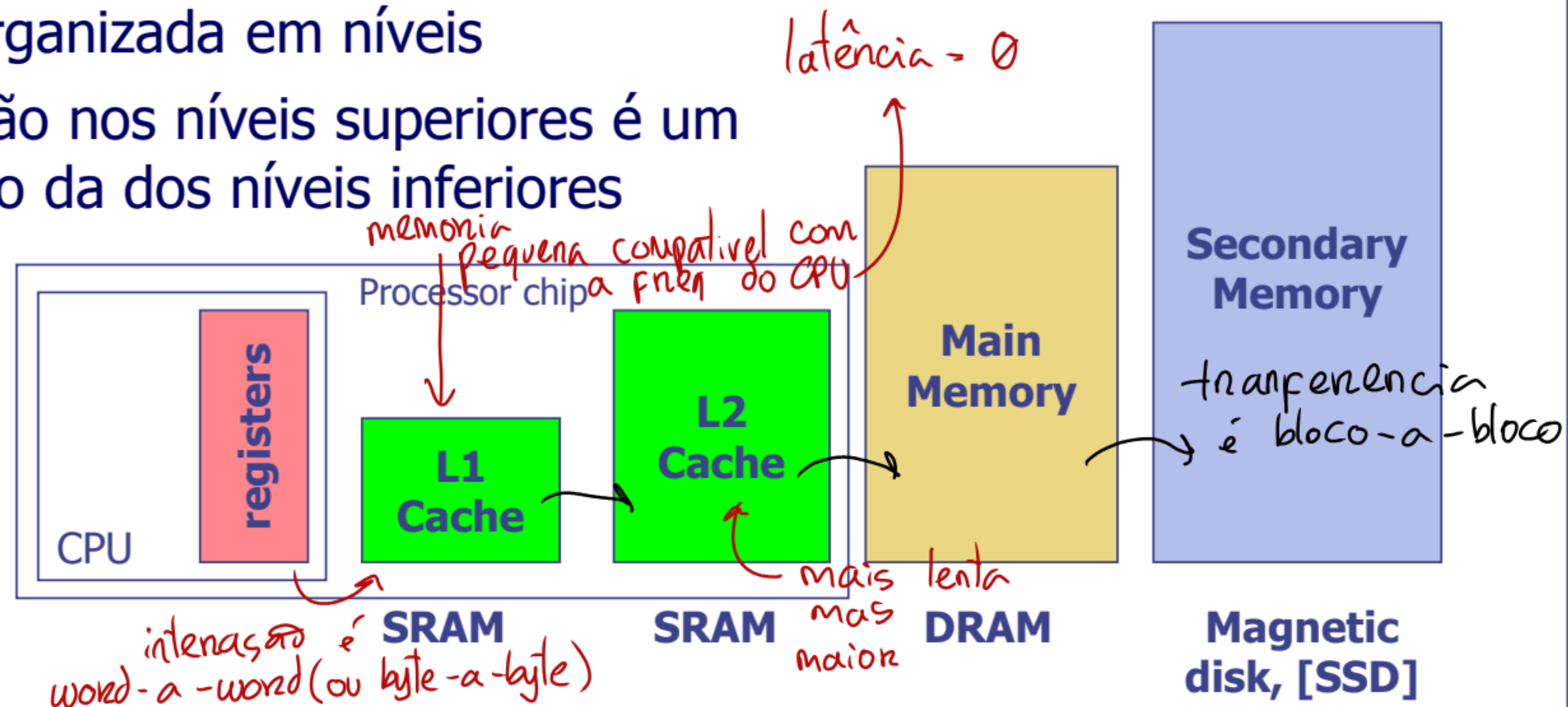


Guardar na SRAM a informação que o CPU precisa mais vezes

Hierarquia de memória

Cada nível tem um subconjunto da informação guardada no nível inferior

- Memória organizada em níveis
- A informação nos níveis superiores é um subconjunto da dos níveis inferiores



Speed:	0.1's nS	1's nS	1's nS	10's nS	10's mS [10's μs]
Size (bytes):	100's	10's K	100's K	1's G	1's T [0.1s T]
\$/byte:	High				Low

capacidade aumenta mas velocidade diminui

- A informação circula apenas entre níveis adjacentes da hierarquia
- **Bloco** – quantidade de informação que circula entre níveis adjacentes (n bytes)

Hierarquia de memória

- Solução 1 – expor a hierarquia *- n é usada normalmente*
 - Alternativas de armazenamento: registos internos do CPU, memória rápida, memória principal, disco
 - Cabe ao programador utilizar racionalmente estas alternativas de armazenamento *n é muito complicado*
 - Exemplo de processador que usa esta técnica: Cell microprocessor (Cell Broadband Engine Architecture; usado por ex. na PlayStation 3 e algumas televisões)
- Solução 2 – esconder a hierarquia *→ Hardware é que sabe em que nível está a informação onde queremos aceder*
 - Modelo de programação:
 - Tipo de memória único
 - Espaço de endereçamento único
 - A máquina gere automaticamente o acesso ao sistema de memória
 - Solução usada na maioria dos processadores contemporâneos

Hierarquia de memória

cache
nivel 1
↑

- A hierarquia de memória combina uma memória adaptada à velocidade do processador (de pequena dimensão) com uma (ou mais) menos rápida, mas de maior dimensão
- A memória rápida armazena um sub-conjunto da informação residente na memória principal
- Uma vez que a memória com que o processador interage diretamente é de pequena dimensão, a eficiência da hierarquia resulta do facto de se copiar informação para a memória rápida poucas vezes e de se aceder a essa informação muitas vezes (antes de surgir a necessidade de a substituir)
- Para se tirar partido deste esquema, a probabilidade de a informação (que o processador necessita) estar nos níveis mais elevados da hierarquia tem que ser elevada
- O que torna essa probabilidade elevada ?

Localidade

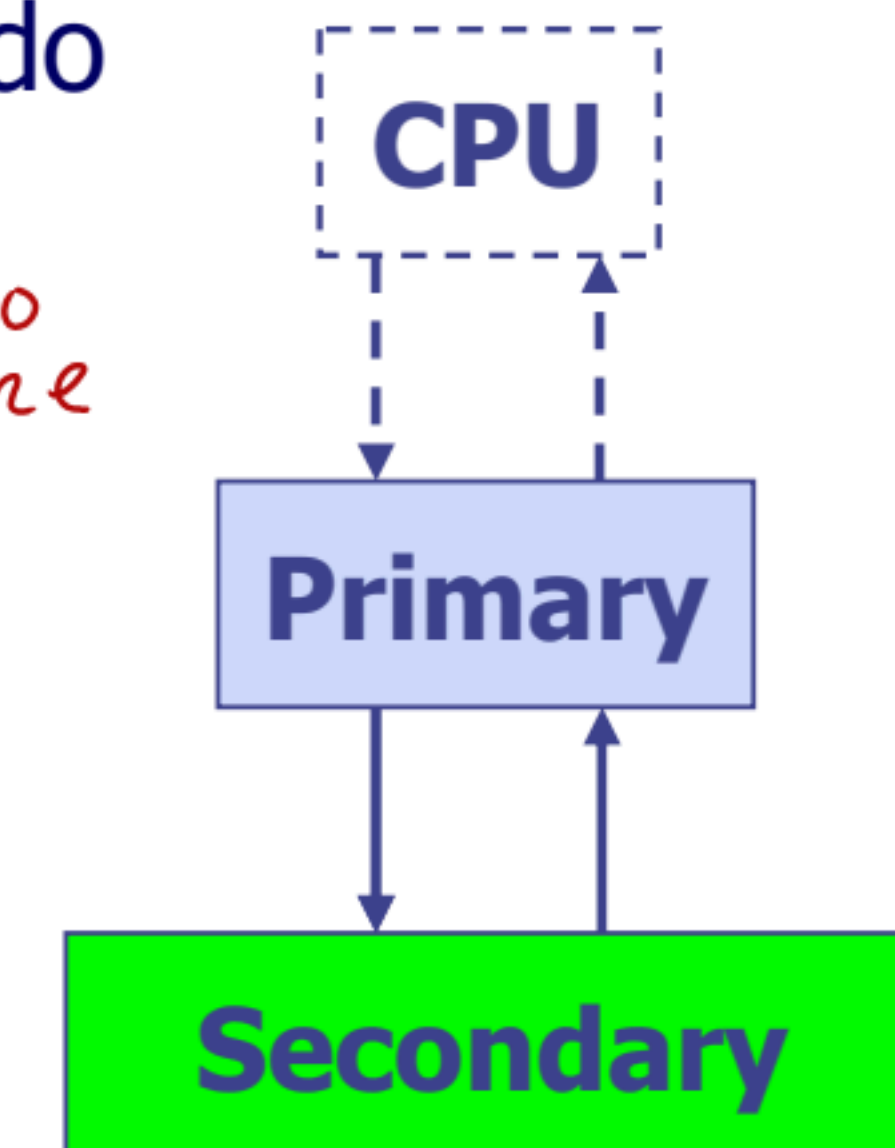
- **Princípio da localidade:** os programas não acedem à memória (dados e instruções) de forma aleatória mas usam tipicamente endereços que se situam na vizinhança uns dos outros
- **Localidade espacial:** → Se o CPU acede a uma posição de memória agora é muito provável que vá querer aceder a uma posição de memória próxima a essa mais tarde
"a informação que o processador necessita de seguida, tem uma elevada probabilidade de estar próxima da que consome agora"
 - Exemplos: instruções de um programa; processamento de um *array*
 - Quanto maior for o bloco (zona contígua de memória) de informação existente na memória rápida, melhor
- **Localidade temporal:**
"a informação que o processador consome agora tem uma elevada probabilidade de ser novamente necessária num curto espaço de tempo"
"i em for (i ...)"
 - Exemplos: variável de controlo de um ciclo, instruções de um ciclo
 - Quantos mais blocos de informação estiverem na memória rápida, melhor

Hierarquia de memória com 2 níveis

- Nível primário (superior) rápido e de pequena dimensão
- Nível secundário (inferior) mais lento mas de maior dimensão
 - O nível primário contém os blocos de memória mais recentemente utilizados pelo CPU
- Os pedidos de informação são sempre dirigidos ao nível primário, sendo o nível secundário envolvido apenas quando a informação pretendida não está nesse nível
 - Se os dados pretendidos se encontram num bloco do nível primário então existe um "hit"
 - Caso contrário ocorre um "miss"
- Na ocorrência de um "miss" acede-se ao nível secundário e transfere-se o bloco que contém a informação pretendida
 - Se o primário tiver cheio ele é primeiro copiado para o secundário

feito pelo hardware

↓ para o nível primário



Hierarquia de memória com 2 níveis

- A taxa de sucesso (**hit ratio**) é dada por:

$$hit_{ratio} = \frac{nr_{hits}}{nr_{accesses}}$$

sempre < 1
(é sempre feito acesso ao disco pelo menos 1 vez)

- A taxa de insucesso (**miss ratio**) é dada por:

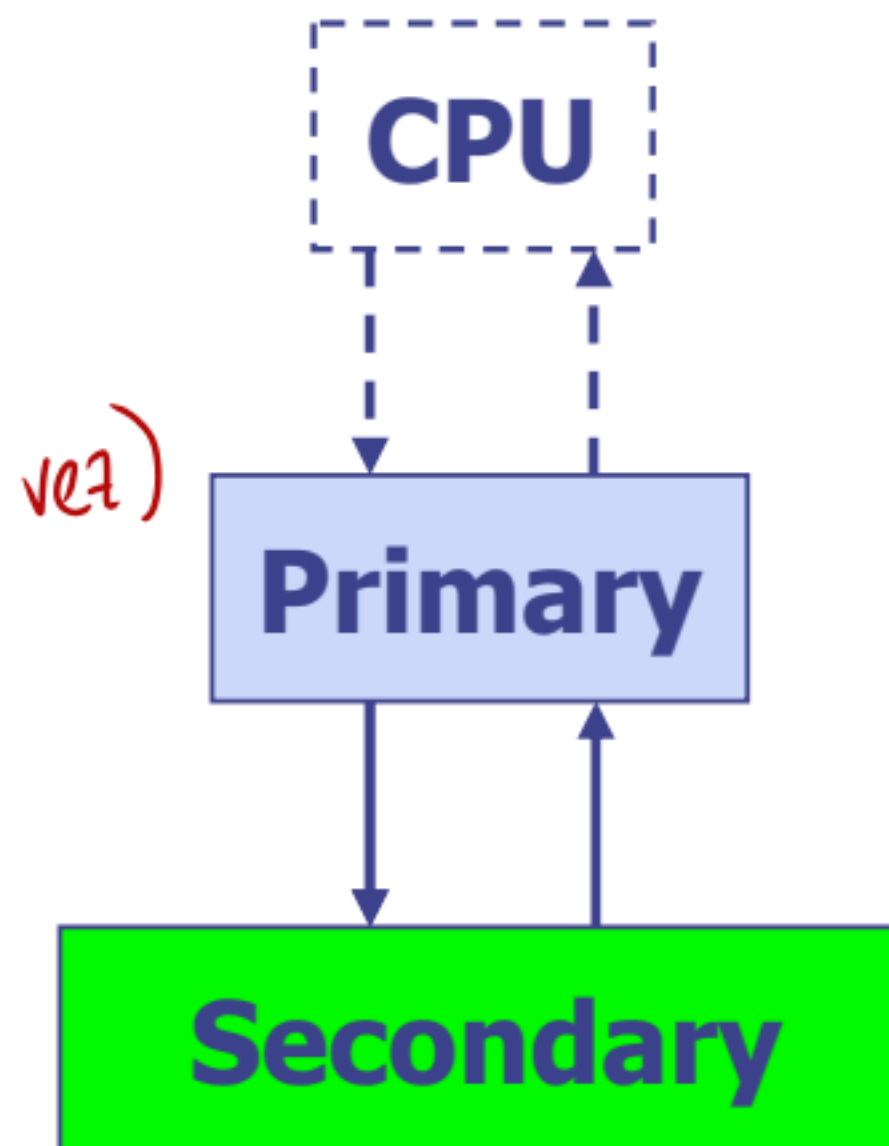
$$miss_{ratio} = 1 - hit_{ratio}$$

- O tempo de acesso no caso de um "hit" designa-se **hit time**
- O tempo de substituir um bloco do nível superior e enviar os dados para o processador é designado por **penalty time (miss penalty)**
- De um modo simplificado, o "penalty time" é dado por:

$$penalty_{time} = hit_{time} + t_{mem}$$

tempo acesso ao nível primário
tempo acesso ao nível secundário

em que " t_{mem} " é o tempo de acesso ao nível secundário



Hierarquia de memória com 2 níveis

- O tempo médio de acesso à informação é então:

$$T_{access} = hit_{ratio} * hit_{time} + (1 - hit_{ratio}) * penalty_{time}$$

$$T_{access} = hit_{ratio} * hit_{time} + (1 - hit_{ratio}) * (hit_{time} + t_{mem})$$

- **Exercício:** Assumindo um *hit_ratio* de 95%, um tempo de acesso ao nível superior de 5ns e um tempo de acesso ao nível inferior de 50ns, calcular o tempo médio de acesso à memória
 - $T_a = 0,95 * 5 + 0,05 * (50 + 5) = 7,5ns$ ← tempo de acesso médio
- Ou seja, aproximadamente 6,7 vezes mais rápido que o acesso direto ao nível secundário
- Quando o acesso a uma palavra no nível superior falha, acede-se ao nível seguinte não apenas a essa palavra, mas também às que estão em endereços adjacentes (vizinhas) - **bloco**

Memória cache

- **cache**: nível de memória que se encontra entre o CPU e a memória principal (primeiras utilizações no início da década de 60)

- Exemplo - **cache read**:

Cache recebe endereço (MemAddr) do CPU
O bloco que contém MemAddr está na cache?

SIM (hit):

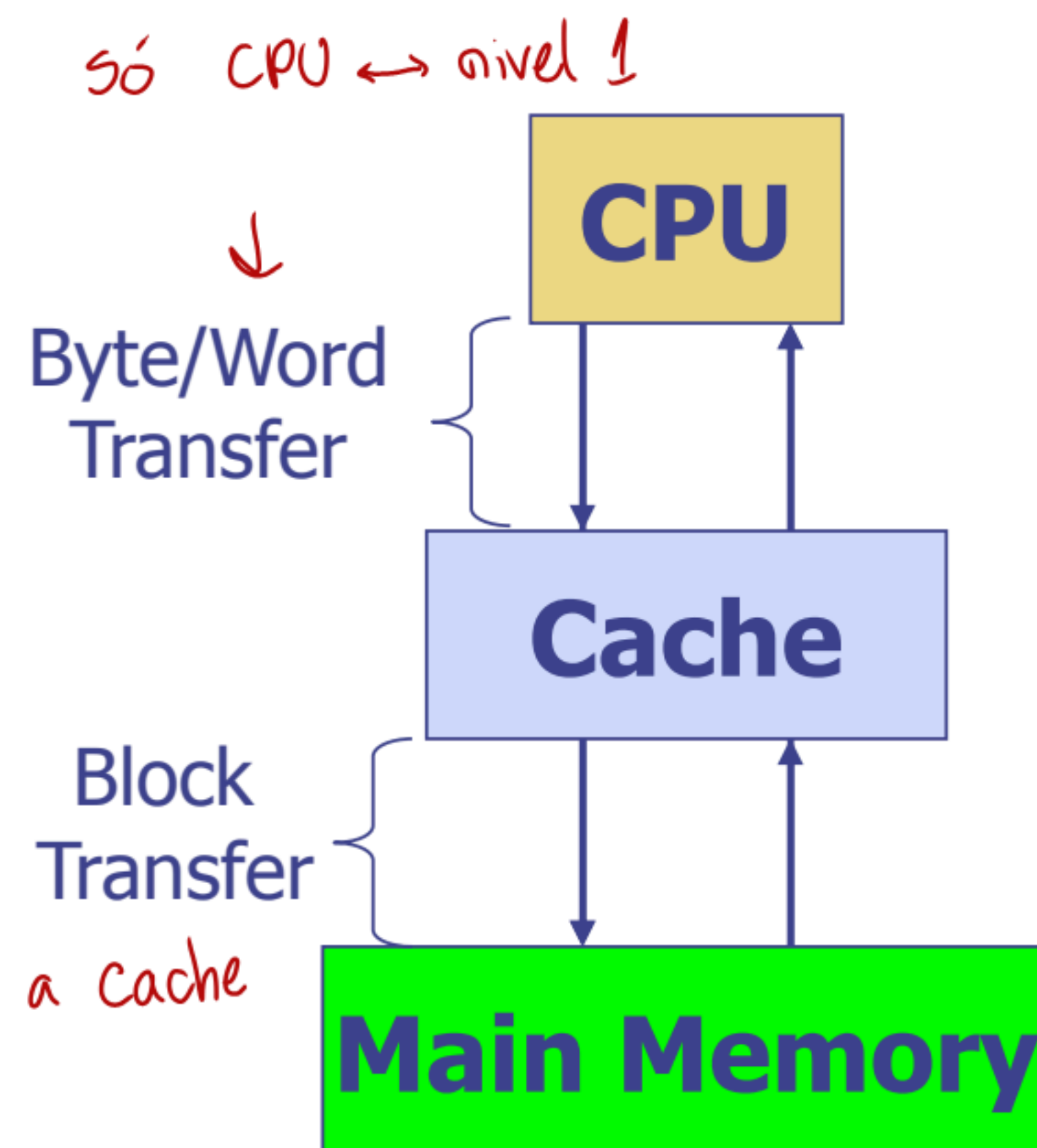
Lê a cache e envia para o CPU o conteúdo de MemAddr

NÃO (miss):

Encontra espaço na cache para um novo bloco

Acede à memória principal e lê o bloco que contém o endereço MemAddr e copia-o para a cache

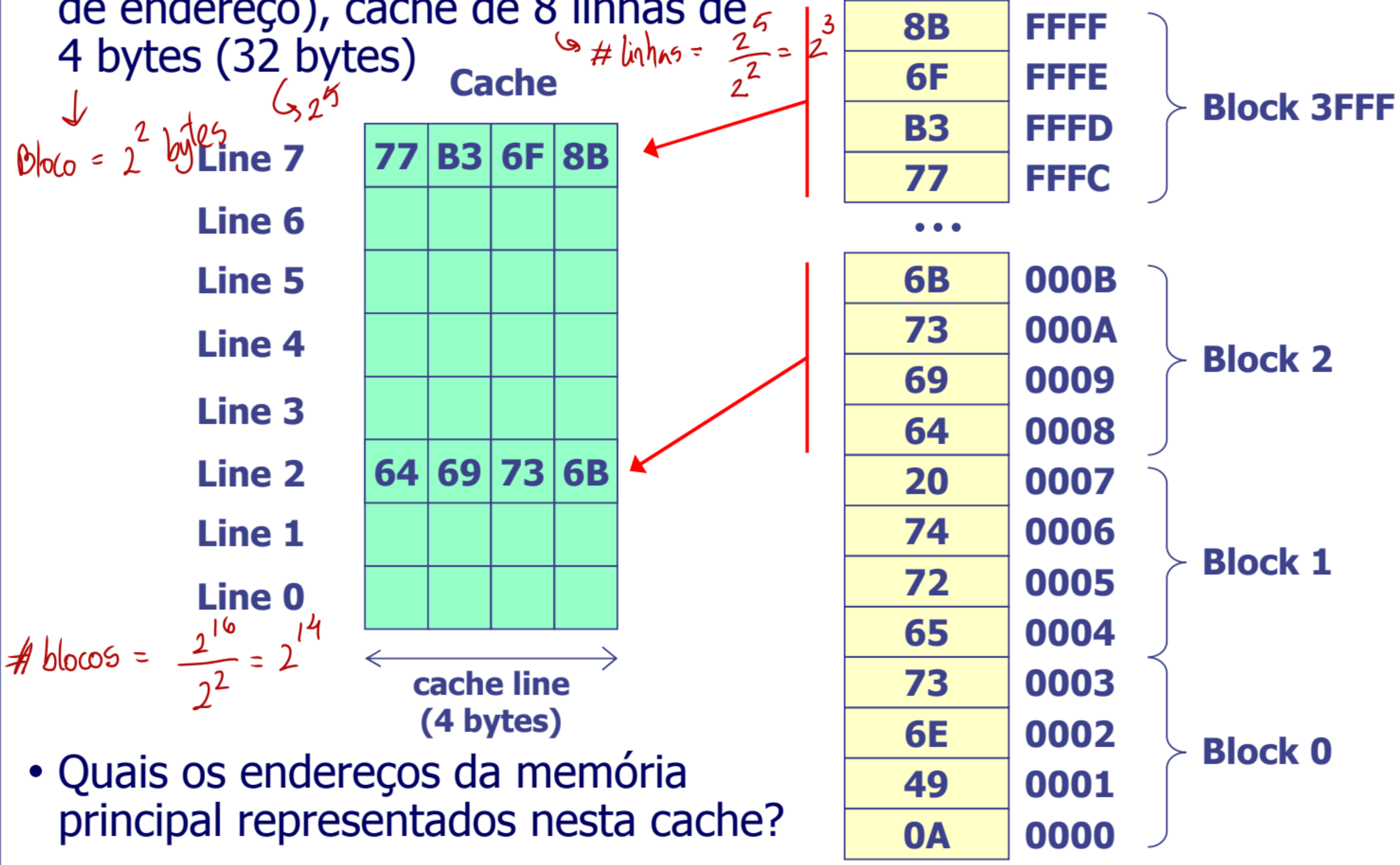
Conteúdo de MemAddr é enviado para o CPU



- É comum os computadores recentes incluírem três níveis de cache, normalmente integrados no mesmo circuito integrado do processador

Memória cache

- Exemplo: memória de 64K (16 bits de endereço), cache de 8 linhas de 4 bytes (32 bytes)



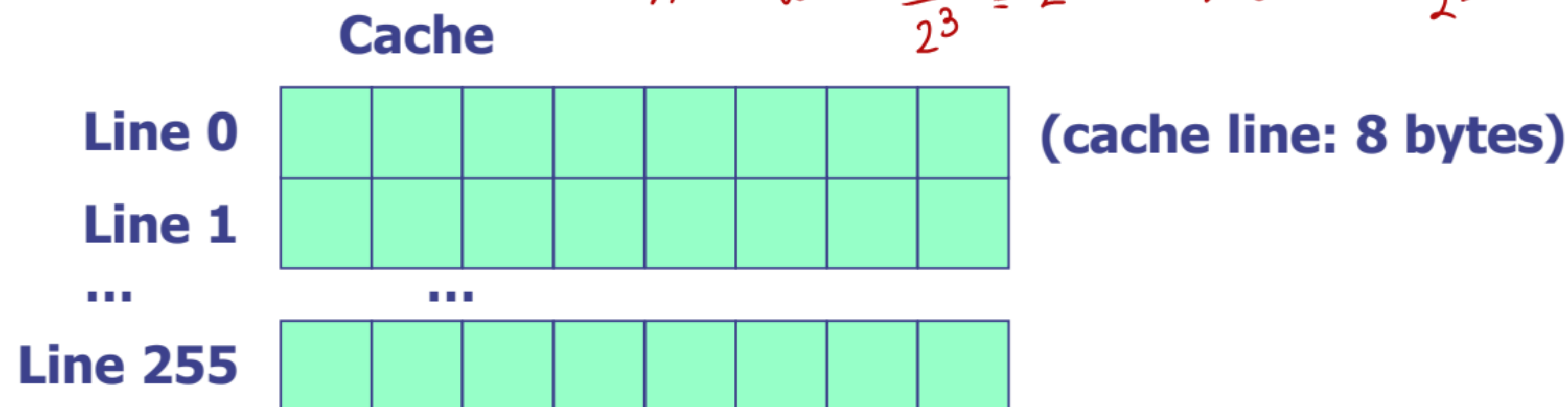
Memória cache

- As operações da cache são transparentes para o processador
- O processador gera um endereço e pede que seja feita uma operação de leitura ou escrita e esse pedido é satisfeito pelo sistema de memória:
 - é desconhecido para o processador se é a cache ou a memória principal a satisfazer o pedido
- Na organização da cache e na implementação da respetiva unidade de controlo é essencial considerar vários aspetos, nomeadamente:
 - Como verificar se um determinado endereço está na cache?
 - Se estiver na cache, onde se encontra?
 - Em caso de *miss*:
 - onde colocar um novo bloco na cache?
 - qual o bloco a retirar da cache?
 - Como lidar com as operações de escrita de modo a garantir a coerência da informação nos vários níveis? *Isso podemos, p.ex., ter uma variável na DRAM incluída na SRAM.*
- A implementação tem de ser eficiente! Realizável em hardware

Organização dos sistemas de cache

- Há basicamente 3 formas de organizar os sistemas de cache:
 - Cache **totalmente associativa** ("fully associative")
 - Cache **com mapeamento direto** ("direct mapped")
 - Cache **parcialmente associativa** ("set associative") ← *mais usada*
- A metodologia de organização de cada uma delas é apresentada a seguir, partindo do seguinte conjunto de especificações-base:
 - Espaço de endereçamento de 16 bits (memória principal organizada em bytes, com 64 kBytes) → 2^{16}
 - Memória cache de 2 kBytes, em que cada linha tem 8 bytes (2^3) (ou seja, cada bloco tem uma dimensão de 8 bytes)

$$\# \text{ linhas} = \frac{2^{16}}{2^3} = 2^8 \quad \# \text{ blocos} = \frac{2^{16}}{2^3} = 2^{13}$$



Organização dos sistemas de cache

- Especificação-base para os exemplos de organização que se seguem:
 - Espaço de endereçamento de 16 bits (0x0000 a 0xFFFF)
 - Memória cache de 2 kBytes, em que cada linha tem 8 bytes (2^3), significa que a cache tem 256 linhas ($2^{11} / 2^3 = 256$)
- Com estes valores, a memória principal pode ser vista como sendo constituída por um conjunto de 2^{13} blocos contíguos, de 8 bytes cada ($2^{16}/2^3 = 2^{13}$): 8k blocos (numerados de 0000 a 0x1FFF)
- Assim, no endereço de 16 bits, os
 - 3 bits menos significativos identificam o **byte dentro do bloco**
 - 13 bits mais significativos identificam o **bloco**

Main memory address:

15	0
13 bits (bloco)	3 bits (byte)

$\# \text{ linhas} = \frac{2^{11}}{2^3} = 2^8$

$\# \text{ blocos} = \frac{2^{16}}{2^3} = 2^{13}$

$\text{Bloco} = 8\text{B} = 2^3$

- Exemplo. Address = 0x001A : **000000000000011010**

Bloco 3, de 0x0018 a 0x001F, byte 2 dentro do bloco

Cache com mapeamento associativo

Cache é preenchida à medida que são lidos blocos da memória

Para a cache saber se um endereço do CPU já está representado na cache

Tag memory

01A5
0077
1FFF
...
0002

Tag
(13 bits)

Valid bit

1
1
1
.
1

Valid
(1 bit)

Cache memory

00	MM block 01A5
01	MM block 0077
02	MM block 1FFF
...	...
FF	MM block 0002

cache line
(8 bytes)

Main memory

MM block 0000
MM block 0001
MM block 0002
...
MM block 0077
...
MM block 01A5
...
MM block 1FFF

Dim. cache line
(8 bytes)

Main memory address:

13 bits (**tag**) | 3 bits (**byte**)

Exemplo:

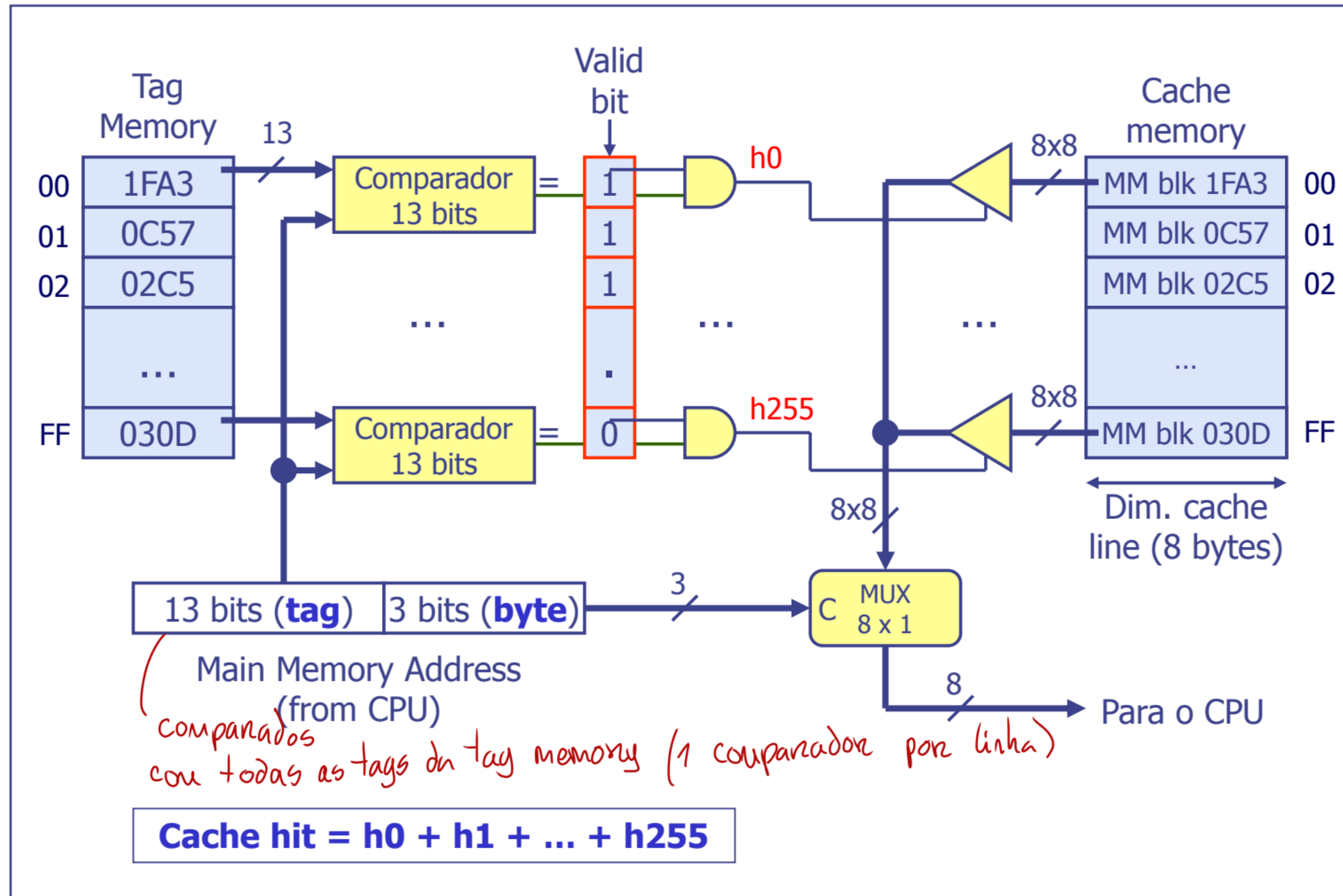
Memory address: 0x0D2A

0000110100101010

-> Tag = 01A5 (Block=0x01A5)

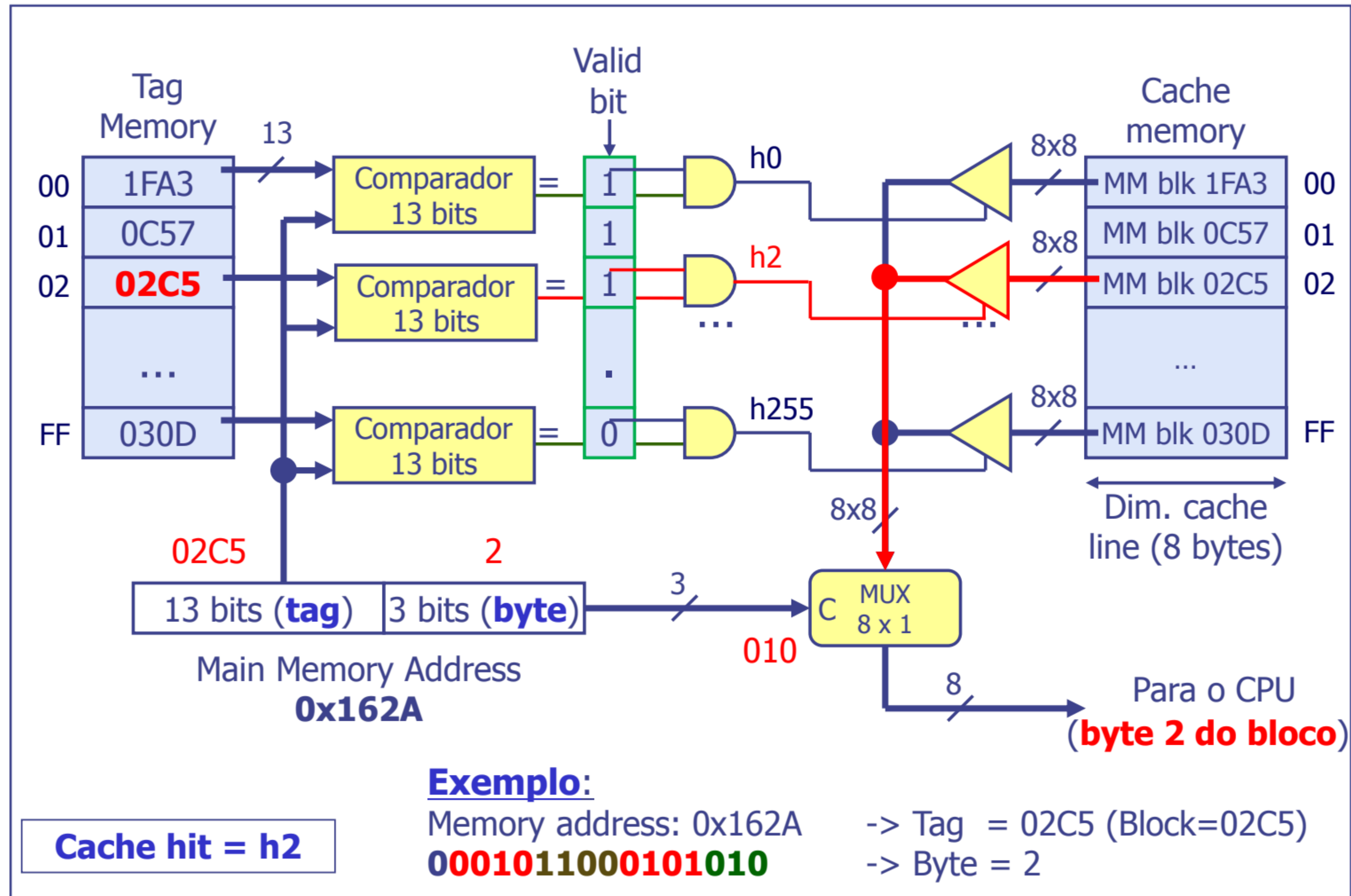
-> Byte = 2

Cache com mapeamento associativo



Cache com mapeamento associativo (exemplo)

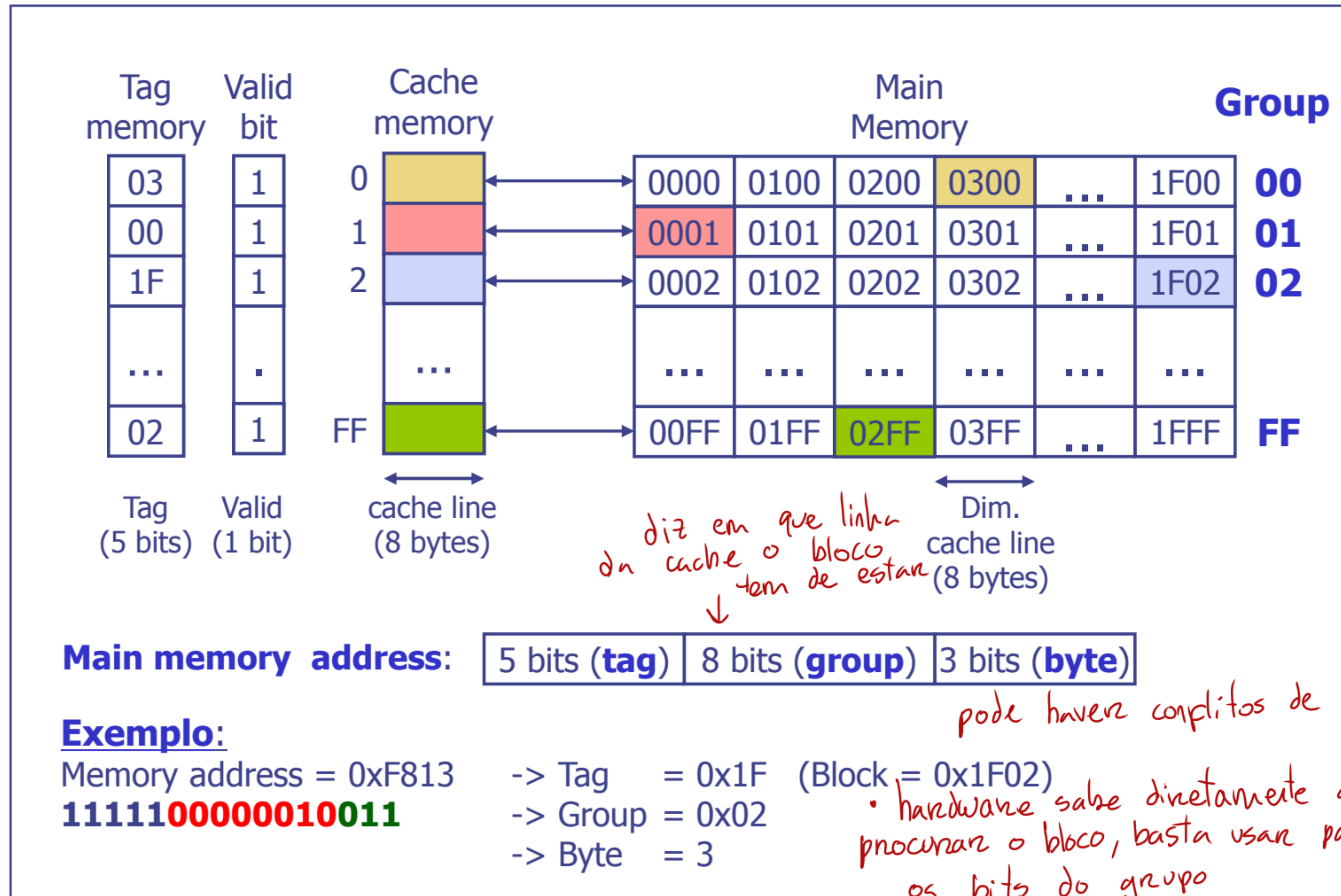
S



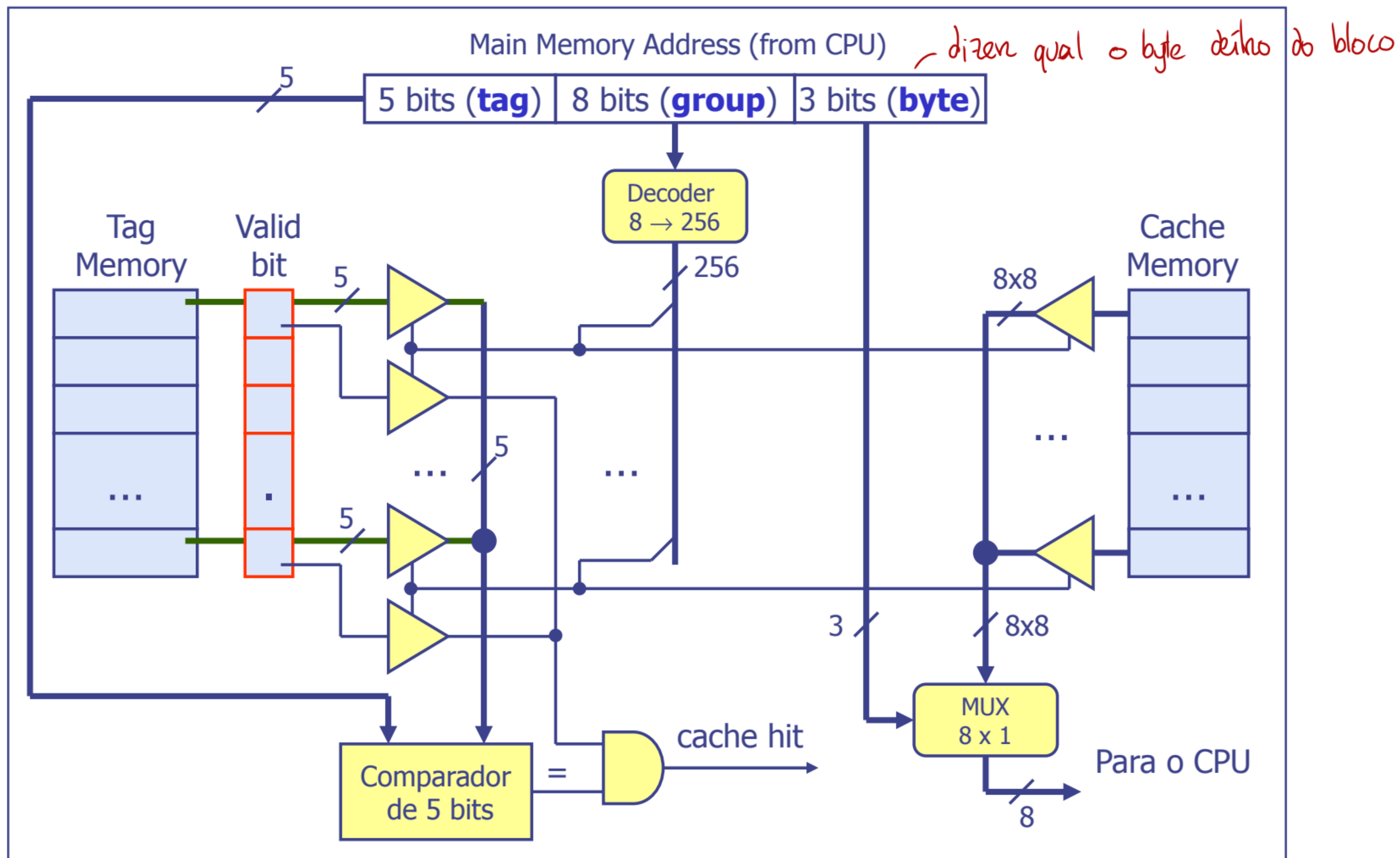
Cache com mapeamento associativo

- Vantagens:
 - Qualquer bloco da memória principal pode ser colocado em qualquer posição da cache
- Inconvenientes:
 - A "tag" tem que ter todos os bits do número do bloco, o que numa implementação realista origina comparadores com uma dimensão elevada
 - Todas as entradas da memória "tag" têm de ser analisadas, de forma a verificar se um endereço se encontra na cache
 - Muitos comparadores, custo elevado

Cache com mapeamento direto *a mais restritiva*



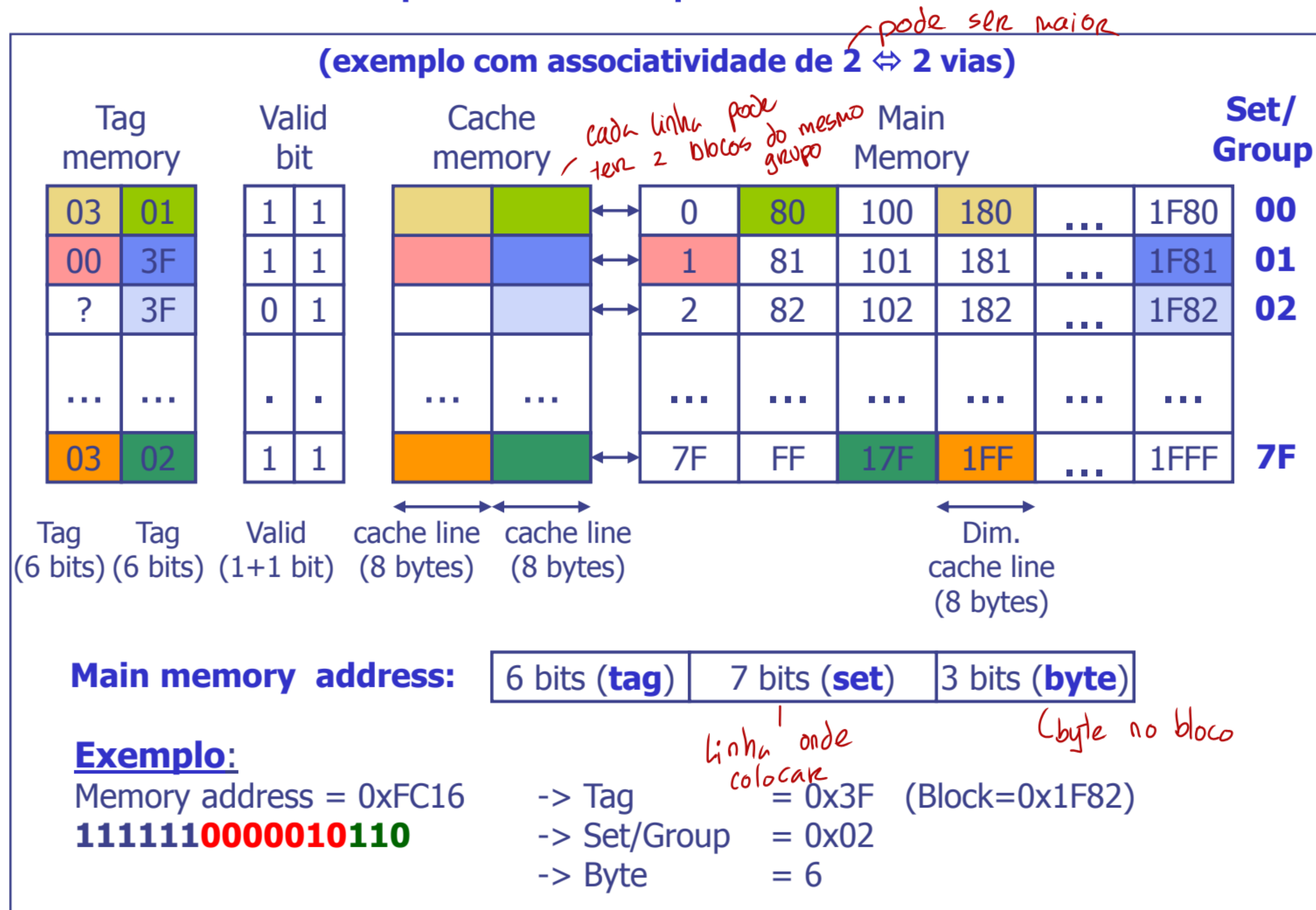
Cache com mapeamento direto



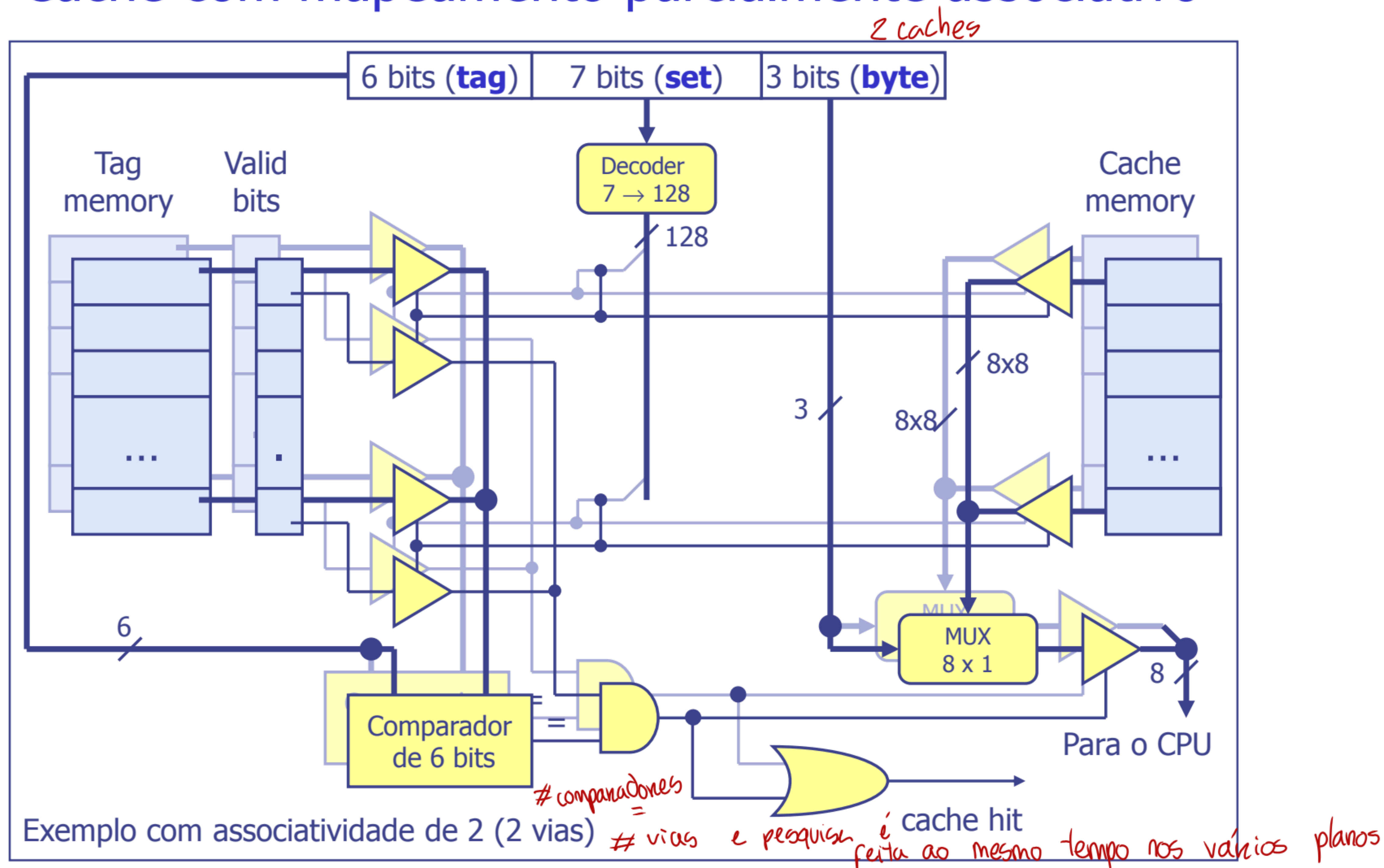
Cache com mapeamento direto

- A cada bloco da memória principal é associada uma linha da cache; o mesmo bloco é sempre colocado na mesma linha
 - Maior vantagem:
 - Simplicidade de implementação
 - Maior desvantagem:
 - Vários blocos têm associada a mesma linha
 - Num dado instante apenas um bloco de um dado grupo pode residir na cache, o que tem como consequência que alguns blocos podem ser substituídos e recarregados várias vezes, mesmo existindo espaço na cache para guardar todos os blocos em uso
 - Uma melhoria óbvia deste tipo de cache seria permitir o armazenamento simultâneo de mais que um bloco do mesmo grupo
 - É esta melhoria que é explorada na cache **parcialmente associativa ("set associative")**
- hardware sabe diretamente onde procurar o bloco, basta usar para os bits do grupo*
- flag menor*

Cache com mapeamento parcialmente associativo



Cache com mapeamento parcialmente associativo



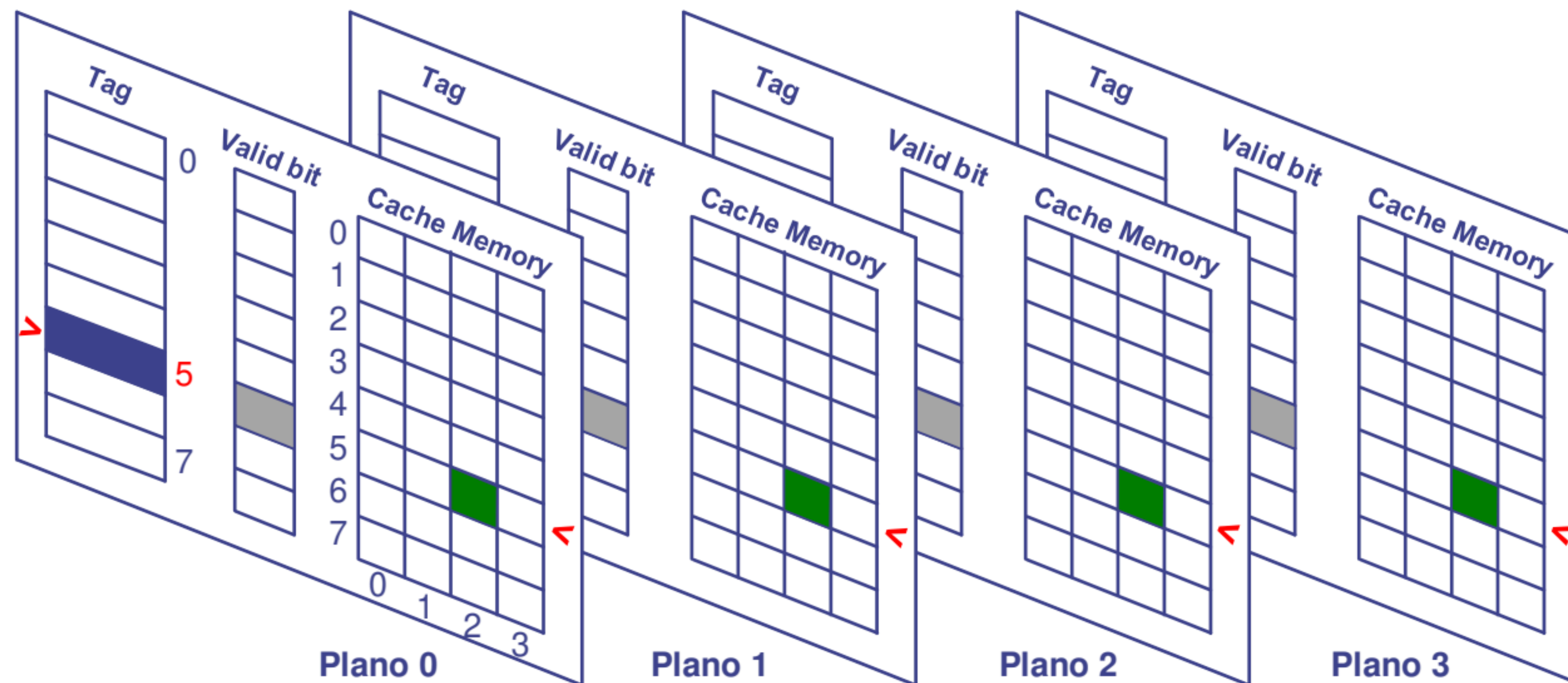
Cache parcialmente associativa

- A cache parcialmente associativa é semelhante à cache com mapeamento direto, mas a primeira permite que mais que um bloco de um mesmo grupo possa ser carregado na cache
- Uma cache com associatividade de 2, ou seja, com 2 vias, permite que 2 blocos do mesmo grupo possam estar simultaneamente na cache = *menos misses*
- A divisão do endereço de memória é igual ao do mapeamento direto (o campo "group" é normalmente designado por "set" - conjunto), mas agora há "n" possíveis lugares onde um dado bloco de um mesmo grupo pode residir; "n" é o número de vias da cache e nesse caso diz-se que a cache tem associatividade de "n"
- As "n" vias onde o bloco pode residir têm que ser procuradas simultaneamente; o hit da cache resulta do OR dos hits de cada uma das vias
- Designações possíveis: Block-set associative cache / n-way-set associative cache

Cache com mapeamento parcialmente associativo



- Esquematização de uma cache com **associatividade de 4**, de **128 bytes**, com **blocos de 4 bytes** ($128 / 4 = 32$ bytes por via, $32 / 4 = 8$ linhas)
- Endereço gerado pelo CPU (16 bits) 0x6A96: **0110101010010110**



- A comparação dos 11 bits mais significativos (A15-A5) do endereço com as "tags" é feita simultaneamente nas 4 vias
- A posição da memória "tag" onde é feita a comparação é determinada pelos bits A4-A2 do endereço (posição 5 no exemplo)
- A via onde ocorre o hit (se ocorrer) fornece o byte armazenado na cache na posição determinada pelos bits A1-A0 do endereço (posição 2 no exemplo)

Políticas de substituição de blocos

Quando é preciso copiar de um nível superior para inferior

- As políticas de substituição de blocos na cache, na ocorrência de um miss, são várias. As mais utilizadas são as seguintes:

- **FIFO** – First in first out: é substituído o bloco que foi carregado há mais tempo

- **LRU – Least Recently Used**: é substituído o bloco da cache que está há mais tempo sem ser referenciado

- **LFU – Least Frequently Used**: substituído o bloco menos

- **Random**: substituição aleatória (testes indicam que não é muito pior do que LRU)

mais usado



→ feito com um bit que é posto a 1 quando o bloco é acessado e de vez em quando todos os bits passam a zero

↑ difícil implementar

(preciso histograma de acesso)

que foi

↓
substituem-se os que têm o bit a zero

Políticas de escrita

- **Write-through** *alterar variável num nível superior também a altera nos outros*
 - Todas as escritas são realizadas simultaneamente na cache e na memória principal
 - Se endereço ausente na cache, atualiza apenas a memória principal (**write-no-allocate**)
 - A memória principal está sempre consistente
- **Write-back** *níveis ficam inconsistentes até os blocos serem substituídos*
 - Valor escrito apenas na cache; novo valor é escrito na memória quando o bloco da cache é substituído
 - "**Dirty bit**" (este bit é ativado quando houver uma escrita em qualquer endereço do bloco presente na linha da cache)
 - Se endereço ausente na cache, carrega o bloco para a cache e atualiza-o (**write-allocate**)
 - Mais complexo do que "write-through"

Arquiteturas podem fazer um mix dos dois

Exemplo1 – Intel Core i7-6500U

CPUs

- Número de cores: 2

- Core 1: L1 data (32 KB), L1 instr (32 KB), L2 data+instr (256 KB)
- Core 2: L1 data (32 KB), L1 instr (32 KB), L2 data+instr (256 KB)
- L3: (4 MB) partilhada pelos 2 cores

Cache	Size	Associativity	Line Size
L1 Data	2 x 32 KB	8-way set associative	64 bytes
L1 Instructions	2 x 32 KB	8-way set associative	64 bytes
L2	2 x 256 KB	4-way set associative	64 bytes
L3	4 MB (shared cache)	16-way set associative	64 bytes

- Qual a dimensão do bloco das caches L1, L2 e L3?
- Qual o número de linhas das caches L1, L2 e L3?

Exemplo2 – Intel Core i7-1165G7

- Número de cores: 4
 - Core 1: L1 data (48 KB), L1 instr (32 KB), L2 data+instr (1.25 MB)
 - Core 2: L1 data (48 KB), L1 instr (32 KB), L2 data+instr (1.25 MB)
 - Core 3: L1 data (48 KB), L1 instr (32 KB), L2 data+instr (1.25 MB)
 - Core 4: L1 data (48 KB), L1 instr (32 KB), L2 data+instr (1.25 MB)
 - L3: (12 MB) partilhada pelos 4 cores

Cache	Size	Associativity	Line Size
L1 Data	4 x 48 KB	12-way set associative	64 bytes
L1 Instructions	4 x 32 KB	8-way set associative	64 bytes
L2	4 x 1.25 MB	20-way set associative	64 bytes
L3	12 MB (shared cache)	12-way set associative	64 bytes

— tamanho do bloco

- Qual a dimensão do bloco das caches L1, L2 e L3?
- Qual o número de linhas das caches L1 (I+D), L2 e L3?

Exercícios

1. Qual a posição dos endereços de bloco 1 e 29 numa cache de mapeamento direto com 8 linhas?
2. Considere um sistema computacional com um espaço de endereçamento de 32 bits e uma cache com 256 blocos de 8 bytes cada um.
 - Qual o tamanho em bits dos campos tag, group e byte supondo que a cache é: 1) associativa; 2) mapeamento direto; 3) com associatividade de 2.
3. Para a implementação de uma cache com 64KB de dados e blocos de 4 bytes num sistema computacional com um espaço de endereçamento de 32 bits, quantos bits de armazenamento são necessários supondo:
 - uma cache associativa;
 - uma cache com mapeamento direto;
 - uma cache com associatividade de 2.